hashlock.

2024

Security Audit

PrimeInsights (DAO)

November 2024 info@hashlock.com.au Hashlock Pty Ltd

Table of Contents

Executive Summary	4
Project Context	4
Audit scope	7
Security Rating	8
Intended Smart Contract Functions	9
Code Quality	10
Audit Resources	10
Dependencies	10
Severity Definitions	11
Audit Findings	12
Centralisation	59
Conclusion	60
Our Methodology	61
Disclaimers	63
About Hashlock	64

CAUTION

THIS DOCUMENT IS A SECURITY AUDIT REPORT AND MAY CONTAIN CONFIDENTIAL INFORMATION. THIS INCLUDES IDENTIFIED VULNERABILITIES AND MALICIOUS CODE WHICH COULD BE USED TO COMPROMISE THE PROJECT. THIS DOCUMENT SHOULD ONLY BE FOR INTERNAL USE UNTIL ISSUES ARE RESOLVED. ONCE VULNERABILITIES ARE REMEDIATED, THIS REPORT CAN BE MADE PUBLIC. THE CONTENT OF THIS REPORT IS OWNED BY HASHLOCK PTY LTD FOR USE OF THE CLIENT.

Executive Summary

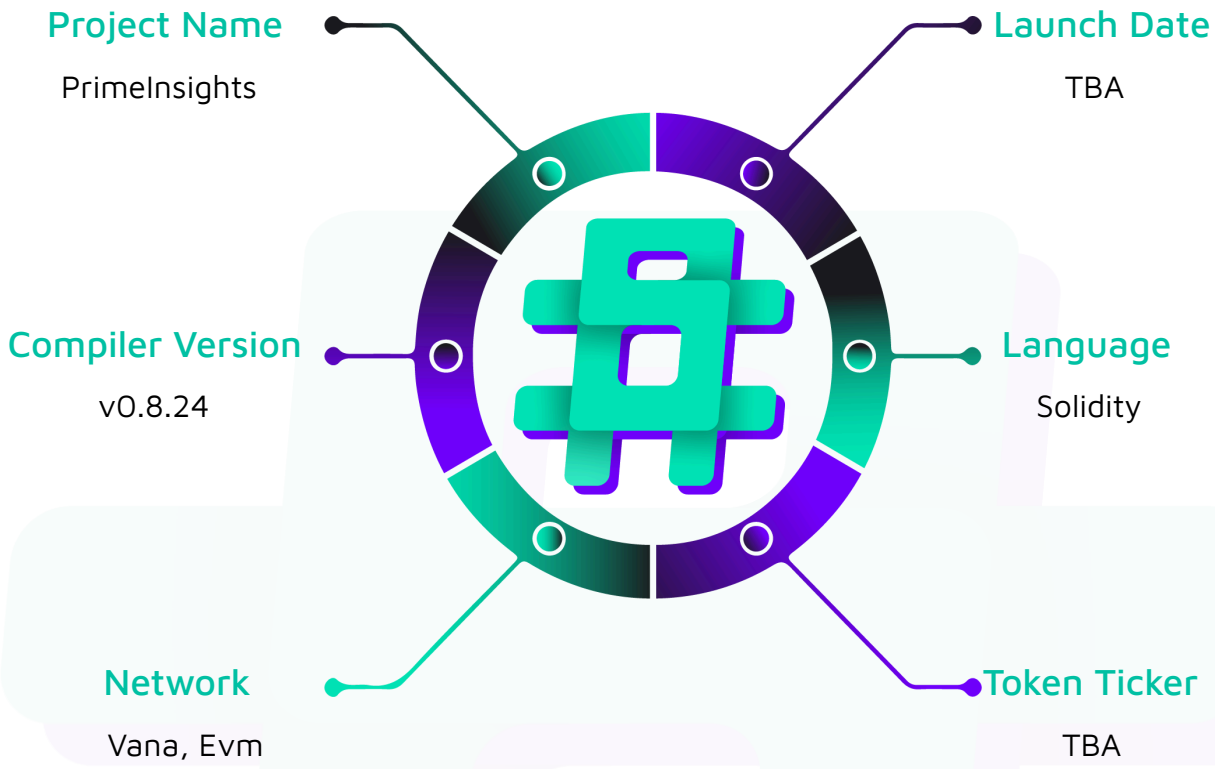
The PrimeInsights team partnered with Hashlock to conduct a security audit of their smart contracts. Hashlock manually and proactively reviewed the code in order to ensure the project's team and community that the deployed contracts are secure.

Project Context

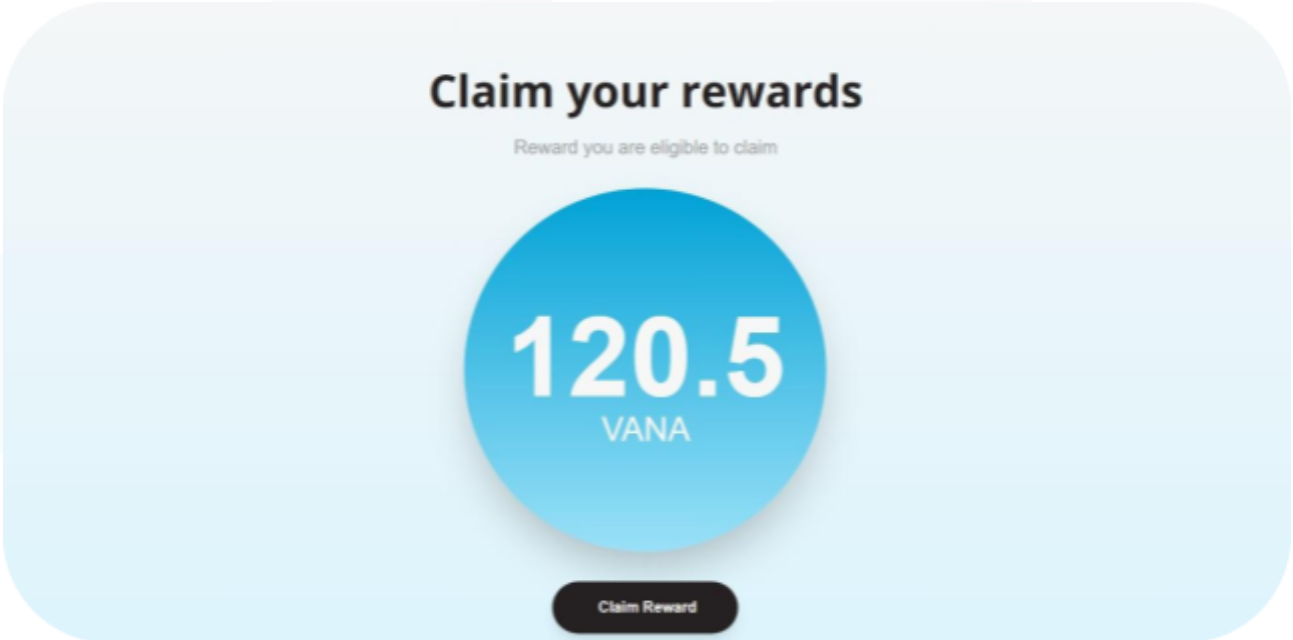
PrimeInsights DAO empowers individuals to reclaim ownership of their Amazon data by integrating it into a decentralized autonomous organization (DAO). Members gain governance rights, enabling them to collectively decide whether to monetize their data by selling it to AI companies or to delete it if the data provider open-sources their models. This innovative approach leverages blockchain technology to ensure transparency and fairness, allowing participants to benefit from the value of their personal data while actively influencing its utilization in the evolving digital economy.

Project Name: PrimeInsights
Compiler Version: ^0.8.24
Website: www.primeinsightsdao.com
Logo:

Visualised Context:



Project Visuals:



Audit scope

We at Hashlock audited the solidity code within the PrimeInsights project, the scope of work included a comprehensive review of the smart contracts listed below. We tested the smart contracts to check for their security and efficiency. These tests were undertaken primarily through manual line-by-line analysis and were supported by software-assisted testing.

Description	PrimeInsights Smart Contracts
Platform	**EVM / Solidity**
Audit Date	**December, 2024**
Contract 1	common_store.sol
Contract 1 MD5 Hash	368742d5831160992ed647d64e5dc547
Contract 2	common.sol
Contract 2 MD5 Hash	772b57ccc79998a43be90bebb3dbc9e5
Contract 3	contributions_store.sol
Contract 3 MD5 Hash	ef8b171e479a509923f4e5b179b0a2b6
Contract 4	contributions.sol
Contract 4 MD5 Hash	6b6c1b4d0237358f27a25d6989c39c2f
Contract 5	data_reg.sol
Contract 5 MD5 Hash	cf8fc644ad33068303926eb82fe43f43
Contract 6	dlp.sol
Contract 6 MD5 Hash	84e3a1232ff12163a4425da83e7ba783
Contract 7	interface.sol
Contract 7 MD5 Hash	466d9f08103ab01e7ee4593715371bd7
Contract 8	permissions_store.sol
Contract 8 MD5 Hash	2dc05ef1844a054f8f2fb81861341b6e

Contract 9	permissions.sol
Contract 9 MD5 Hash	d9eca947264094b1baf5721c86472678
Contract 10	rewards_store.sol
Contract 10 MD5 Hash	7f4f1664e23d69f6845571f8ec42ad8a
Contract 11	rewards.sol
Contract 11 MD5 Hash	027a75e1e39b233d1c8e0bbbcfac8a7e
Contract 12	scoring_store.sol
Contract 12 MD5 Hash	87b72beef2083e0170d25aaed52ed530
Contract 13	scoring.sol
Contract 13 MD5 Hash	20bcc41fae80a36183791201d8a041da
Contract 14	tee.sol
Contract 14 MD5 Hash	013574d45d1ebc09ab0a4692c5da2655
GitHub Commit Hash	0e6dc748cb1e740787fff89f094e61bd7c50416c

Security Rating

After Hashlock's Audit, we found the smart contracts to be **"Secure"**. The contracts all follow simple logic, with correct and detailed ordering. They use a series of interfaces, and the protocol uses a list of Open Zeppelin contracts. We initially identified some significant vulnerabilities that have since been addressed.

The 'Hashlocked' rating is reserved for projects that ensure ongoing security via bug bounty programs or on chain monitoring technology.

All issues uncovered during automated and manual analysis were meticulously reviewed and applicable vulnerabilities are presented in the Audit Findings section. The general security overview is presented in the Standardised Checks section and the project's contract functionality is presented in the Intended Smart Contract Functions section.

All vulnerabilities initially identified have now been resolved and acknowledged.

Hashlock found:

1 Low severity vulnerability

1 Gas Optimisation

2 QAs

Caution: *Hashlock's audits do not guarantee a project's success or ethics, and are not liable or responsible for security. Always conduct independent research about any project before interacting.*

Intended Smart Contract Functions

Claimed Behaviour	Actual Behaviour
common.sol - Allows UPDATE role to: - Set a public key, proof instruction, and the file reward factor	**Contract achieves this functionality.**
contributions.sol - Allows users to: - Add/Remove contributions	**Contract achieves this functionality.**
data_reg.sol - Allows REGISTRY role to: - Update the data registry	**Contract achieves this functionality.**

interface.sol
- Allows users to:
  - Add files with permissions
  - Request rewards
- Allows PAUSE role to:
  - Pause/Unpause the contract
	Contract achieves this functionality.

rewards.sol
- Allows EDIT role to:
  - Add/Remove the reward tokens
- Allows users to:
  - Add rewards to the contract
  - Claim rewards
	Contract achieves this functionality.

scoring.sol - Allows EDIT role to: - Enable/Disable categories	**Contract achieves this functionality.**

Code Quality

This audit scope involves the smart contracts of the PrimeInsights project, as outlined in the Audit Scope section. All contracts, libraries, and interfaces mostly follow standard best practices and to help avoid unnecessary complexity that increases the likelihood of exploitation, however, some refactoring was required.

The code is very well commented on and closely follows best practice nat-spec styling. All comments are correctly aligned with code functionality.

Audit Resources

We were given the PrimeInsights project smart contract code in the form of Github access.

As mentioned above, code parts are well commented. The logic is straightforward, and therefore it is easy to quickly comprehend the programming flow as well as the complex code logic. The comments are helpful in providing an understanding of the protocol's overall architecture.

Dependencies

As per our observation, the libraries used in this smart contracts infrastructure are based on well-known industry standard open source projects.

Severity Definitions

Significance	Description
High	High-severity vulnerabilities can result in loss of funds, asset loss, access denial, and other critical issues that will result in the direct loss of funds and control by the owners and community.
Medium	Medium-level difficulties should be solved before deployment, but won't result in loss of funds.
Low	Low-level vulnerabilities are areas that lack best practices that may cause small complications in the future.
Gas	Gas Optimisations, issues, and inefficiencies
QA	Quality Assurance (QA) findings are purely informational and don't impact functionality. These notes help clients improve the clarity, maintainability, or overall structure of the code, ensuring a cleaner and more efficient project. They should be addressed for optimization but are not critical to the system's performance or security.

Audit Findings

Low

[L-01] common#setRewardSender, setName, setPublicKey, setProofInstruction, tee#updateTeePool, data_reg#updateDataRegistry - Lack of emitting events

Description

The above functions are missing events when the key variables are updated.

Recommendation

Add relevant events in the functions.

Status

Resolved

Gas

[G-01] rewards#_addRewardToken, isRewardTokenActive, scoring#calculateTotalScoreForContribution - Cache array length before looping

Description

The functions have a loop and read the length from storage at every iteration.

Before iterating through an array with a for-loop, the length of the array can be cached in memory so that its value does not need to be accessed on each iteration of the loop.

Recommendation

Cache the array length into a memory variable and use it in the loops.

Status

Resolved

QA

[Q-01] Contracts - Floating pragma

Description

The contracts use `pragma solidity ^0.8.24`.

This can allow the contracts to be deployed with the wrong pragma version which is different from the one the contracts were tested with.

Recommendation

Lock the pragma version.

Status

Resolved

[Q-02] contributions#removeContribution - Unnecessary check

Description

The `removeContribution` function has a check if the msg.sender is `address(0)`. However, the caller can never be the zero address and this check is not needed.

Recommendation

Remove the unnecessary check.

Status

Resolved

Centralisation

The PrimeInsights project values security and utility over decentralisation.

The owner executable functions within the protocol increase security and functionality but depend highly on internal team responsibility.

Conclusion

After Hashlock's analysis, the PrimeInsights project seems to have a sound and well-tested code base, now that our vulnerability findings have been resolved and acknowledged. Overall, most of the code is correctly ordered and follows industry best practices. The code is well commented on as well. To the best of our ability, Hashlock is not able to identify any further vulnerabilities.

Our Methodology

Hashlock strives to maintain a transparent working process and to make our audits a collaborative effort. The objective of our security audits is to improve the quality of systems and upcoming projects we review and to aim for sufficient remediation to help protect users and project leaders. Below is the methodology we use in our security audit process.

Manual Code Review:

In manually analysing all of the code, we seek to find any potential issues with code logic, error handling, protocol and header parsing, cryptographic errors, and random number generators. We also watch for areas where more defensive programming could reduce the risk of future mistakes and speed up future audits. Although our primary focus is on the in-scope code, we examine dependency code and behaviour when it is relevant to a particular line of investigation.

Vulnerability Analysis:

Our methodologies include manual code analysis, user interface interaction, and white box penetration testing. We consider the project's website, specifications, and whitepaper (if available) to attain a high-level understanding of what functionality the smart contract under review contains. We then communicate with the developers and founders to gain insight into their vision for the project. We install and deploy the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, skim open issue tickets, and generally investigate details other than the implementation.

Documenting Results:

We undergo a robust, transparent process for analysing potential security vulnerabilities and seeing them through to successful remediation. When a potential issue is discovered, we immediately create an issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is vast because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We generally follow a process of first documenting the suspicion with unresolved questions, and then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this, we analyse the feasibility of an attack in a live system.

Suggested Solutions:

We search for immediate mitigations that live deployments can take and finally, we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinised by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the contract details are made public.

Disclaimers

Hashlock's Disclaimer

Hashlock's team has analysed these smart contracts in accordance with the best industry practices at the date of this report, in relation to: cybersecurity vulnerabilities and issues in the smart contract source code, the details of which are disclosed in this report, (Source Code); the Source Code compilation, deployment, and functionality (performing the intended functions).

Due to the fact that the total number of test cases is unlimited, the audit makes no statements or warranties on the security of the code. It also cannot be considered as a sufficient assessment regarding the utility and safety of the code, bug-free status, or any other statements of the contract. While we have done our best in conducting the analysis and producing this report, it is important to note that you should not rely on this report only. We also suggest conducting a bug bounty program to confirm the high level of security of this smart contract.

Hashlock is not responsible for the safety of any funds and is not in any way liable for the security of the project.

Technical Disclaimer

Smart contracts are deployed and executed on a blockchain platform. The platform, its programming language, and other software related to the smart contract can have their own vulnerabilities that can lead to attacks. Thus, the audit can't guarantee the explicit security of the audited smart contracts.

About Hashlock

Hashlock is an Australian-based company aiming to help facilitate the successful widespread adoption of distributed ledger technology. Our key services all have a focus on security, as well as projects that focus on streamlined adoption in the business sector.

Hashlock is excited to continue to grow its partnerships with developers and other web3-oriented companies to collaborate on secure innovation, helping businesses and decentralised entities alike.

Website: hashlock.com.au
Contact: info@hashlock.com.au